Hidden in the Leftovers

Task-Specific Reconstruction Gaps of Sparse Autoencoders
in English and Indian Languages

Prakhar Singhal (2022111025, prakhar.singhal@research.iiit.ac.in)
Raveesh Vyas (2022114007, raveesh.vyas@research.iiit.ac.in)

Mechanistic Interpretability of Language Models (CL3.412)
Course project proposal · Team IpCp-- · October 2026

Summary. Sparse autoencoders (SAEs) are widely used to interpret language-model activations and are usually evaluated by average reconstruction fidelity, on which the Gemma Scope SAEs score 98–99%. In a pilot study on gemma-2-2b, we measure with activation patching how much of a task's causal effect the SAE features carry. On object tracking at layer 12 they recover 74% of the effect; one direction of the SAE error, fitted on that task, recovers about half of the remainder, while directions from other tasks or generic text recover almost none. We propose to test how general such task-specific gaps are across task families, whether they reflect the model's binding mechanism, and whether they are larger in Hindi and Hinglish.

1 What: research question and hypothesis

A language model processes text as **tokens** (words or word pieces) through a stack of **layers**; at each layer it holds an activation vector x (2304 numbers per token for gemma-2-2b). A **sparse autoencoder** (SAE) writes $x = \hat{x} + \varepsilon$, where $\hat{x}$ is reconstructed from a small number of interpretable **features** and ε is the **SAE error**, the part it does not explain (also called "dark matter" [4]; informally, the leftovers). A **direction** is a fixed vector in this 2304-dimensional space. Appendix D defines all terms.

Research question. *When SAE features fail to carry information a model is using for a task, is the missing part specific to that task, and is more of it missed in Indian languages?*

Hypothesis. SAE reconstruction gaps are of two kinds. Some are *concentrated and task-specific*: one or two directions of the error, fitted on that task, recover about half of the gap (object tracking at layer 12; the running sum at layer 18). Others are *diffuse*, spread over many directions (the names task at late layers). Average fidelity measures neither. We ask:

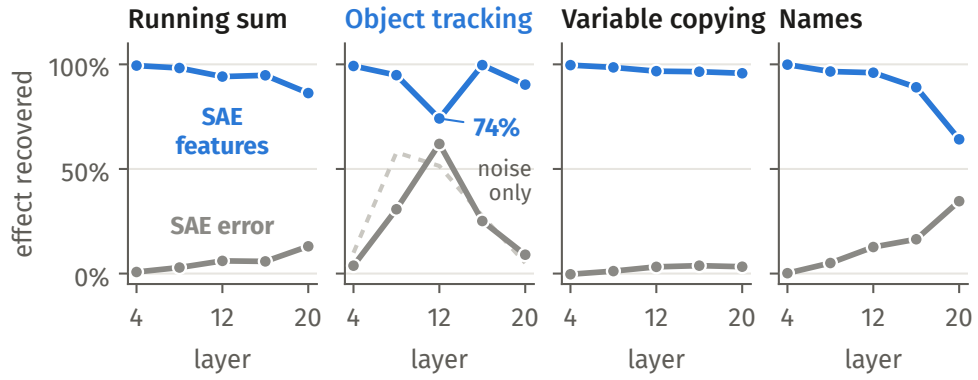
- **Q1. Generality.** Which task families show a concentrated, task-specific gap, and is it tied to a prompt template or to a type of computation?
- **Q2. Mechanism.** In entity-tracking tasks, is the error direction part of the model's *binding* mechanism, which associates an entity with its attribute [5]?
- **Q3. Language.** Are the gaps larger in Hindi and Hinglish than in English, and do the same directions recover them?

2 Why: motivation

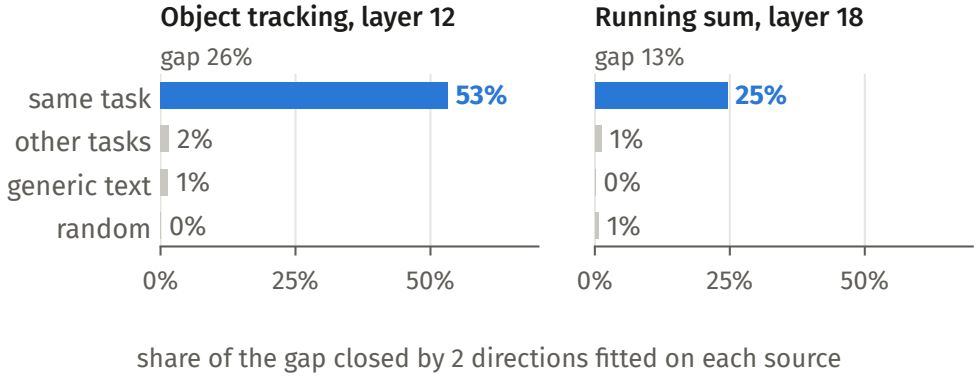
SAEs are used to explain, audit and steer models, and public suites such as Gemma Scope [9] are evaluated mainly by reconstruction fidelity: the fraction of next-token performance retained when $\hat{x}$ replaces x. For the 16k Gemma Scope SAEs at layers 10 to 18 this is 98% to 99% on generic text. Fidelity is an average and does not test any particular computation. The SAE error is known to be large and partly structured [4] and to matter in feature circuits [11]. If the information SAEs miss were generic, a general remedy (wider SAEs, or a correction fitted on generic text) would address all tasks. If it is task-specific, a feature-based explanation of a task can be incomplete in ways that fidelity does not reveal, and a per-task causal check is needed. The question extends to language: the Gemma Scope SAEs were trained on predominantly English text [9]. Prior work identifies language-specific and shared SAE features [1, 2] but does not measure the causal role of the error per language.

3 What has been done: pilot study

Setup (Appendix B). Model: gemma-2-2b, an open 2-billion-parameter model [6]. SAEs: Gemma Scope residual-stream SAEs, 16k (16,384) to 262k features. Tasks: a running sum ("Start with 2.



(a) Patching $\hat{x}$ recovers most of the effect; patching ε recovers little (dashed: ε replaced by noise, object tracking).



(b) Two error directions fitted on the same task close a substantial share of the gap; directions from other tasks, generic text or random directions close almost none.

Figure 1: Pilot results (16k SAEs, 150 test pairs). Full results in Appendix B.4.

Add 3. Add 1. Add 2. Total =" → 8), object tracking [8] ("The key is in box B … The key moves to box C … The key is in box" → C), variable copying ("t = 7. q = 4. x = q. q = 2. x =" → 4), and the indirect-object (names) task as a control [14]. Three algorithmic tasks planned originally were dropped because the model performed at shortcut level on them (Appendix A). **Activation patching** [15]: prompts A and B differ in one step, so their answers differ. We run the model on B and, at one layer, replace part of B's activation with A's, either $\hat{x}$ or ε; the **effect recovered** (NLDR) is the resulting shift of the output from B's answer towards A's (0 = none, 1 = full, as when all of x is patched). Directions are fitted on 300 training pairs; all scores use 150 held-out test pairs.

1. **Features carry most of the effect.** On the three tracking tasks, patching $\hat{x}$ recovers 74% to 100% at every layer tested (4 to 20); patching ε recovers 0% to 13%, except object tracking at layers 8 to 16 (25% to 62%), which is largely explained by a noise control that gives 28% to 58% (Figure 1a). The names control loses more at late layers (24% to 36% at layers 18 to 20).
2. **A concentrated gap.** For object tracking at layer 12, $\hat{x}$ recovers 74% (95% CI 73% to 76%). Adding the top principal direction of the training-set error raises this to 88%, about half of the gap; a random direction adds nothing, and 32 directions fitted on generic text add at most 1 point.
3. **Task specificity.** Two directions fitted on the same task close 39% to 53% of that gap (two fitting methods); directions from other tasks close −1% to 2%, and from generic text 1% (Figure 1b). The running sum at layer 18 (gap 13%) shows the same pattern (25% to 58% vs at most 1%). The names gap is diffuse: about 32 directions are needed to close half of it at layer 18.
4. **Width.** From 16k to 262k features the object-tracking gap is 26%, 20%, 24%, 23% and 15%; one error direction closes 51% to 69% of the remainder at every width. The wider SAEs also have more active features per token (78 vs 124), and the reduction narrowly misses a persistence criterion fixed before the run (Appendix B.4); width therefore remains an open question for Q1.
5. **Methodological pitfalls.** Three known pitfalls would each have produced a misleading result: patching only the last token, probe accuracy without a prompt-only baseline [7], and equating decodability with causal use (Appendix B.5).

Limitations. One model, one SAE family and synthetic tasks; width was tested at one layer; low-rank interventions can produce interpretability illusions [10], which our random, generic-text and held-out controls address only in part.

4 How: plan and success criteria

Q1, generality. Tasks grouped by computation: *entity binding* (objects in boxes; people and cities, e.g. "Asha lives in Pune. Ravi lives in Delhi. Asha lives in"), *counting* (longer running sums), *reference following* (variable copying) and the names control. For each task, layer and width we measure the gap $(1 - \mathrm{NLDR}(\hat{x}))$, the number of error directions needed to close half of it, and a **specificity score** (share closed by same-task directions minus the largest share from any other source). Transfer between the two binding tasks distinguishes a template-specific from a computation-specific gap; comparing SAEs at matched numbers of active features separates width from sparsity.

Q2, mechanism. Binding-ID vectors were reported for LLaMA and Pythia models [5]. We re-derive them for gemma-2-2b, compare them with the error direction, and intervene along that direction to test whether the model's answer moves to the other entity's box.

Q3, language. The same tasks in Hindi (Devanagari), Hinglish (Latin script, code-mixed), Tamil or Bengali where the model can perform them, and French as a non-Indian control. Because Hinglish also introduces English words, we add Hindi sentences in Latin script without English words. Example (*draft, to be validated by a native speaker*): शुरुआत 2 से करें। 3 जोड़ें। 1 जोड़ें। 2 जोड़ें। कुल $= \rightarrow 8$. Fidelity per language uses IndicCorp v2 [3] and FineWeb-2 [12]. Design in Appendix C.

Table 1: Success criteria, fixed in advance (95% confidence intervals throughout).

	Supported if	Not supported if
Q1	gap $\geq 10\%$, and same-task directions close $\geq$ 30 points more of it than any other source	on ≥ 2 new tasks with a gap, other sources close as much as same-task directions
Q1 transfer	directions from one binding task close $\geq$ half as much of the other's gap as its own directions	transfer near zero (template-specific)
Q2	intervening along the direction moves $\geq 50\%$ of pairs to the other box; a random direction $\leq 5\%$	the intervention is no more effective than a random direction
Q3	Hindi fidelity ≥ 3 points lower and a larger gap; the English direction closes $\geq$ half as much as Hindi's own	Hindi's gap is not larger than English's

Deliverables. (1) Gap size, type and specificity for each task, layer, width and language tested. (2) The patching code, with the three pitfalls handled. (3) A result on the binding hypothesis. **Resources and risks.** One 8 GB GPU and the tested pilot pipeline (the three main pilot runs took about 6.3 hours). If binding vectors cannot be re-derived for gemma-2-2b, Q2 is limited to the intervention test; if the model cannot perform the tasks in Tamil or Bengali, we report this and keep Hindi and Hinglish; answers must be single tokens, so letters or digits replace names where needed.

5 When: timeline

When	Step	Output
Oct 19–26 (done)	Pilot study, four stages (Appendix B.2)	Concentrated gap identified
Week 1 (Oct 27–Nov 2)	Q2: binding vectors for gemma-2-2b, intervention; Q1: new tasks, matched-sparsity widths	Q2 result
Week 2 (Nov 3–9)	Q1: gap and specificity per task; Q3: task performance in Hindi and Hinglish	Q1 table; usable languages
Weeks 3–4 (Nov 10–23)	Q3: fidelity, gap and transfer per language; Tamil or Bengali if feasible	Q3 table
Week 5 (Nov 24–30)	Buffer; additional test pairs; robustness checks	Final figures
Week 6 (Dec 1–7)	Report and final presentation	Submission

References

[1] J. Brinkmann et al. Large language models share representations of latent grammatical concepts across typologically diverse languages. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6131–6150, Apr. 2025. Association for Computational Linguistics.

[2] B. Deng et al. Unveiling language-specific features in large language models via sparse autoencoders. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 4563–4608, July 2025. Association for Computational Linguistics.

[3] S. Doddapaneni et al. Towards leaving no Indic language behind: Building monolingual corpora, benchmark and models for Indic languages. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12402–12426, July 2023. Association for Computational Linguistics.

[4] J. Engels, L. R. Smith, and M. Tegmark. Decomposing the dark matter of sparse autoencoders. *Transactions on Machine Learning Research*, 2025.

[5] J. Feng and J. Steinhardt. How do language models bind entities in context? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[6] Gemma Team et al. Gemma 2: Improving open language models at a practical size. *arXiv preprint arXiv:2408.00118*, 2024.

[7] J. Hewitt and P. Liang. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2733–2743, Nov. 2019. Association for Computational Linguistics.

[8] N. Kim and S. Schuster. Entity tracking in language models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3835–3855, July 2023. Association for Computational Linguistics.

[9] T. Lieberum et al. Gemma Scope: Open sparse autoencoders everywhere all at once on Gemma 2. In *Proceedings of the 7th BlackboxNLP Workshop: Analyzing and Interpreting Neural Networks for NLP*, pages 278–300, Nov. 2024. Association for Computational Linguistics.

[10] A. Makelov et al. Is this the subspace you are looking for? an interpretability illusion for subspace activation patching. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[11] S. Marks et al. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models. In *The Thirteenth International Conference on Learning Representations (ICLR)*, 2025.

[12] G. Penedo et al. FineWeb2: One pipeline to scale them all — adapting pre-training data processing to every language. In *Second Conference on Language Modeling (COLM)*, 2025.

[13] S. Rajamanoharan et al. Jumping ahead: Improving reconstruction fidelity with JumpReLU sparse autoencoders. *arXiv preprint arXiv:2407.14435*, 2024.

[14] K. R. Wang et al. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

[15] F. Zhang and N. Nanda. Towards best practices of activation patching in language models: Metrics and methods. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

Appendices

The original plan and every detail of the pilot study

Roadmap. A: the original plan and why its tasks were dropped. B: the pilot study: setup, stages, measures, all results, pitfalls, verdict on the original hypotheses, caveats. C: multilingual design. D: glossary.

A The original position

The project began as *PRISM* (Probing Residual Information and State Manifolds), a test of a strong version of the "dark matter" hypothesis: that SAEs fail to represent the step-by-step state a model computes, so that this state resides in the SAE error. It made three hypotheses, each tied to one experiment (Table 2).

Table 2: The original hypotheses and how each was to be tested.

	Hypothesis	Experiment	Prediction if true
H1	The SAE error carries the task state	*Causal split:* copy $\hat{x}$ or ε from prompt A into a run on prompt B	copying ε brings back more of the answer than copying $\hat{x}$
H2	The state forms a low-dimensional structure in the error	*Geometry:* the spread of ε across directions, probes for the state, low-dimensional projections	a probe reads the state from ε better than from $\hat{x}$ and better than from the prompt's words
H3	A few error directions repair the SAE	*Repair:* $\hat{x}$ plus A's error along its top r directions	a small r closes most of the gap; random directions do not

Original tasks. Three algorithmic tasks whose answer depends on a hidden running state: a sum with a remainder (modular arithmetic), the depth of nested brackets, and the size of the graph reachable from a node, plus the names task as a control. All share one format: prompts A and B that differ in one step, with a one-token answer.

Why they were dropped. The first overnight run (October 19) kept no usable pairs. A capability study (October 20; 128 pairs per setting, 0 to 8 worked examples, several difficulty levels) showed that gemma-2-2b does no better than simple shortcuts on all three: at the planned difficulty, accuracy was 13% vs a 21% shortcut (remainder sums), 41% vs 41% (bracket depth) and 30% vs 48% (graph size). Easier versions did not help: remainder sums and bracket depth stayed flat, and graph size improved but never became usable. The names control reached 100% with two worked examples, so the pipeline was sound. Three replacement tasks passed, alongside the names control (Figure 2).

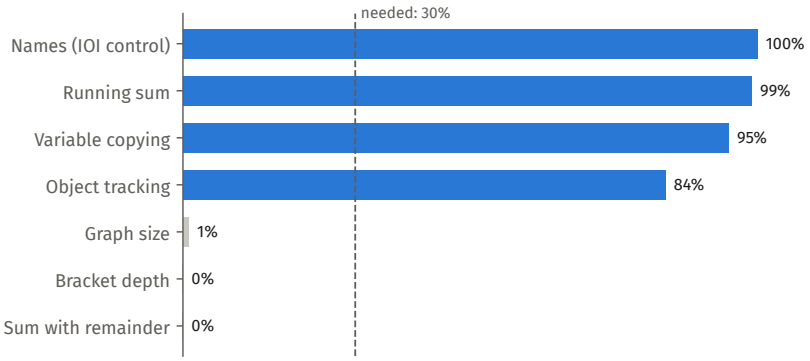


Figure 2: gemma-2-2b can do the four tasks we use, not the three first planned. Share of usable pairs in the capability study (model correct, and changing one step changes its answer; 30% needed). Used tasks at the difficulty we run, dropped tasks at their best setting.

B The pilot study

B.1 Setup

Table 3: Pilot setup.

Model	gemma-2-2b: 26 layers, internal signal of 2304 numbers per token, 16-bit (bf16) precision
SAEs	Gemma Scope residual-stream SAEs with a JumpReLU activation [9, 13] (`gemma-scope-2b-pt-res-canonical`), widths 16k, 32k, 65k, 131k, 262k
Fidelity	16k SAEs at layers 10, 12, 14, 18 keep 99.1%, 98.7%, 98.5%, 98.0% of next-word performance on ordinary text (the Pile), measured against replacing the signal with its average
Pairs	300 training and 150 test pairs per task and setting, no prompt shared; all fitting on training pairs only
Pair filter	running sum, variable copying, names: the model must give both answers correctly and with a clear margin; object tracking (harder for the model): A's answer must be correct and the one-step change must shift the model's preference by a clear margin
Statistics	95% paired bootstrap confidence intervals
Code checks	78 unit tests; matches Hugging Face's implementation of the model (same top answer on 95% of prompts)
Compute	one 8 GB laptop GPU (RTX 5060)

B.2 Stages

Table 4: The four stages of the pilot.

Date	Stage	What ran	Outcome
Oct 19	0. Overnight run	original three tasks, all three experiments	no usable pairs; no results
Oct 20	1. Checks	code checked against Hugging Face; capability study of 7 tasks	original tasks dropped; three replacements chosen
Oct 25	2. Quick pilot	24 min; 160 training / 80 test pairs; layers 6, 12, 18; 16k SAEs	H1 rejected; a small, concentrated gap in object tracking and the running sum
Oct 26	3. Main pilot	three runs, about 6.3 h: patching at layers 4 to 20; repair and specificity at layers 12 to 18; widths 16k to 262k at layer 12	results below

The quick pilot and the main runs agree: object tracking at layer 12 gave 75% vs 74% (features) and 61% vs 62% (error), and one direction closed 51% vs 52% of the gap; the running sum's gap at layer 18 was 12% vs 13%.

B.3 Measures

Effect recovered (NLDR, normalized logit-difference recovery). A *logit* is the model's raw score for a possible next token. With D = logit of A's answer minus logit of B's answer, NLDR = $(D_{\text{copied}} - D_B)/(D_A - D_B)$, averaged over test pairs: 0 = nothing recovered, 1 = fully. **Copy conditions:** the whole signal (control, must give 1); $\hat{x}$ only (A's $\hat{x}$, B's ε); ε only (A's ε, B's $\hat{x}$); noise (B's $\hat{x}$ plus random noise of the same size as ε, which erases B's error). **Positions:** copying every token from the first one that differs (used everywhere except pitfall 1) or the last token only. **Repair:** $\hat{x}$ plus A's error projected on r directions U_r, i.e. $\hat{x} + U_r U_r^\top \varepsilon$. Directions come from the task's own training errors (*unsupervised*: top principal directions; *supervised*: directions that best separate the answers), from ordinary text, from another task (*transfer*), or at random. **Gap** = 1 − NLDR($\hat{x}$ only); **share of the gap closed** = (repair − $\hat{x}$ only)/gap. **Probes:** simple linear classifiers trained to read the answer from the signal at the last token, compared with a probe that sees only the prompt's words.

B.4 Results

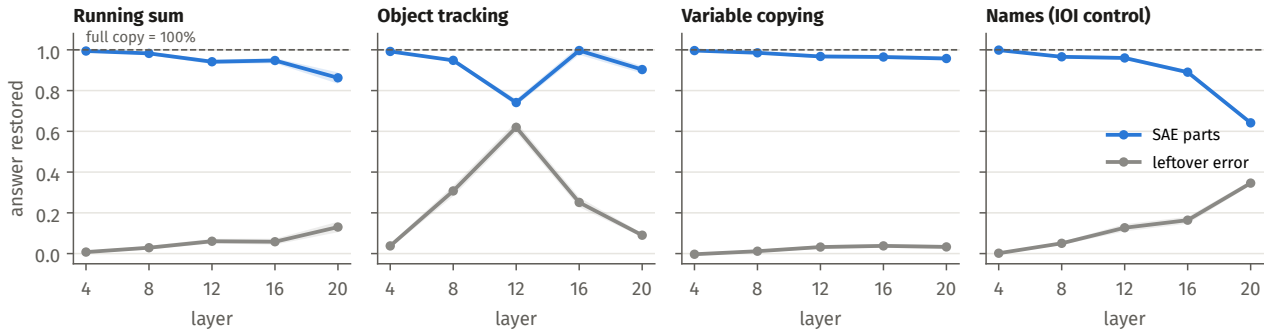


Figure 3: Patching the features ("SAE parts", blue) recovers most of the effect at every layer; patching the error (grey) little, except object tracking in the middle layers (copy from the first changed token; 95% intervals).

Table 5: Effect recovered (%) by patching the features / the error, per layer.

	layer 4	layer 8	layer 12	layer 16	layer 20
Running sum	99 / 1	98 / 3	94 / 6	95 / 6	86 / 13
Object tracking	99 / 4	95 / 31	74 / 62	100 / 25	90 / 9
Variable copying	100 / 0	99 / 1	97 / 3	96 / 4	96 / 3
Names (control)	100 / 0	97 / 5	96 / 13	89 / 16	64 / 35

Object tracking's error-only score is inflated: replacing the error with noise alone gives 28% to 58% at layers 8 to 16 (51% at layer 12), because it erases B's error; A's error adds about 11 points at layer 12.

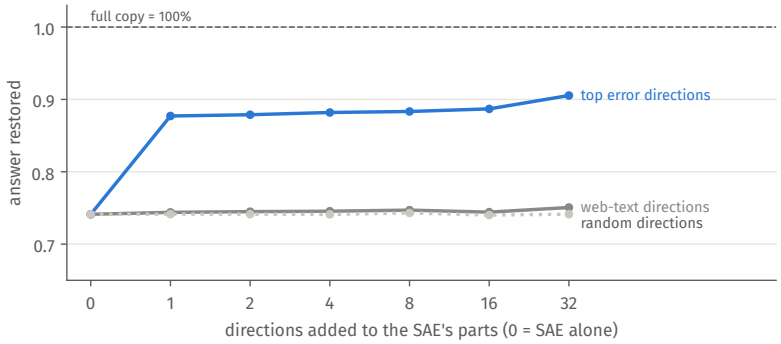


Figure 4: Object tracking at layer 12: one error direction lifts 74% to 88%; directions learned on ordinary text ("web text") or at random add nothing, even 32 of them.

Table 6: Object tracking, layer 12: effect recovered (%) by $\hat{x}$ plus N directions.

N	0	1	2	4	8	16	32
top error directions	74	88	88	88	88	89	91
ordinary-text directions	74	74	74	75	75	74	75
random directions	74	74	74	74	74	74	74

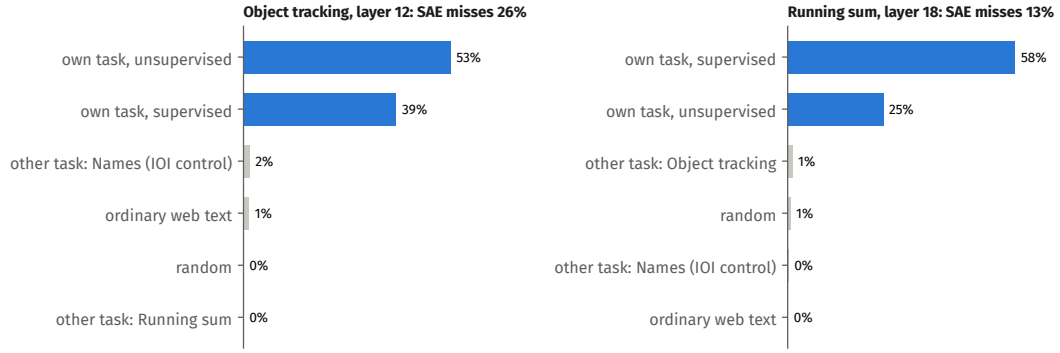


Figure 5: Only directions learned from the same task close a real share of the gap (2 directions per source, 16k SAE; "ordinary web text" is the Pile).

Table 7: Share of the gap closed (%) by 2 directions from each source.

Task (gap)	Source	Closed
Object tracking, layer 12 (26%)	same task, unsupervised / supervised	53 / 39
	other task: names / running sum	2 / −1
	ordinary text / random	1 / 0
Running sum, layer 18 (13%)	same task, supervised / unsupervised	58 / 25
	other task: object tracking / names	1 / 0
	ordinary text / random	0 / 1

The names control is the other kind of gap: it loses 24% at layer 18 (36% at layer 20), and about 32 same-task directions are needed to close half of it at layer 18.

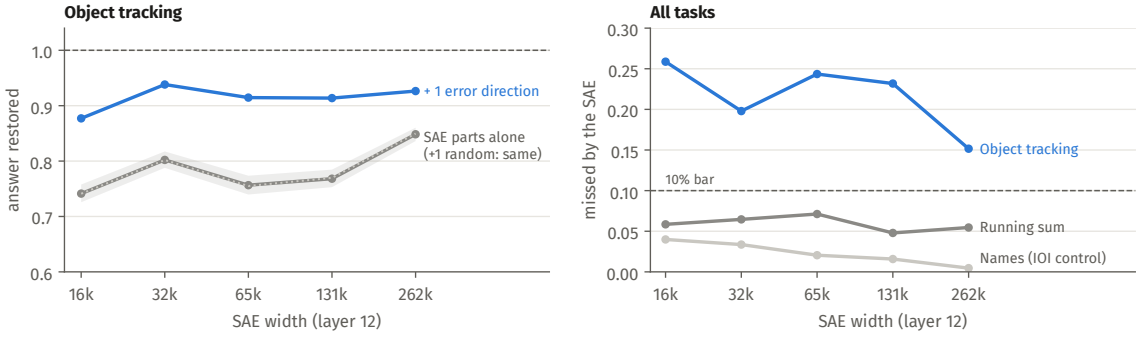


Figure 6: Wider SAEs at layer 12. Left: object tracking with the features alone (grey) and with one error direction (blue). Right: the gap for three tasks.

Table 8: Width sweep at layer 12 (%). "Active" = features active per token.

Task	Width	$\hat{x}$ only [95% CI]	gap	+1 error dir.	+1 random dir.	active
Object tracking	16k	74 [73, 76]	26	88	74	78
	32k	80 [79, 82]	20	94	80	75
	65k	76 [74, 77]	24	91	76	76
	131k	77 [75, 78]	23	91	77	131
	262k	85 [84, 86]	15	93	85	124
Running sum	16k / 262k	94 / 95	6 / 5	94 / 94	94 / 95	80 / 129
Names (control)	16k / 262k	96 / 100	4 / 0	96 / 100	96 / 100	89 / 139

Across all widths the running sum's gap stays between 5% and 7% and the names gap at or below 4%. Before the run we set a rule: a gap counts as persistent if the widest SAE keeps at least 60% of

the 16k gap. Object tracking keeps 15.2% of 25.9%, just under the 15.5% bar, so by our own rule the width result is borderline; we test it again with sparsity matched (Q1).

B.5 Three pitfalls

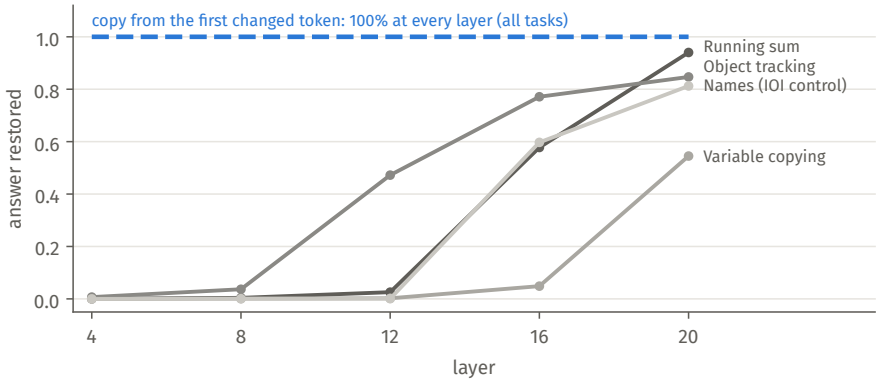


Figure 7: Pitfall 1: copying the whole signal at the last token only brings back little in early and middle layers; copying from the first changed token brings back 100% at every layer.

Pitfall 1, copying only the last token. In early and middle layers the state still sits on earlier tokens, so copying only the last token brings back little even with the *whole* signal (%, layers 4/8/12/16/20): running sum 0/0/3/58/94; object tracking 1/4/47/77/85; variable copying 0/0/0/5/55; names 0/0/0/60/81. Copying from the first changed token brings back 100% at every task and layer [cf. 15].

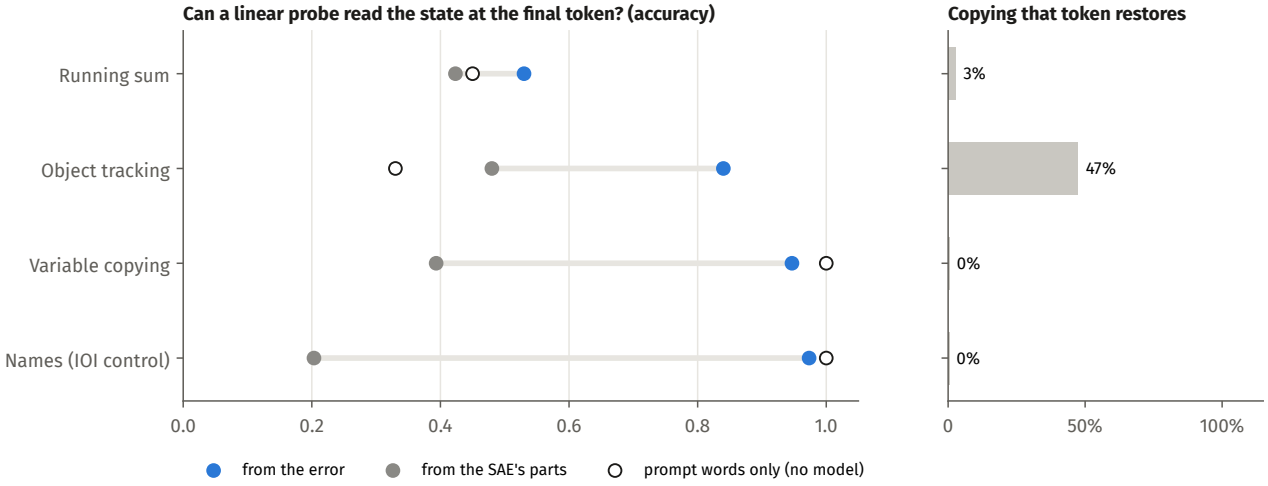


Figure 8: Pitfalls 2 and 3 at layer 12: probes read the state from the error better than from the features, but the prompt's words alone often do as well, and copying that token brings back little.

Table 9: Probe accuracy at the last token, layer 12 (%), and what copying that token brings back.

	probe on ε	probe on $\hat{x}$	words only	chance	last-token copy
Running sum	53	42	45	25	3
Object tracking	84	48	33	18	47
Variable copying	95	39	100	11	0
Names (control)	97	20	100	11	0

Pitfall 2, the prompt gives it away: a probe that sees only the prompt's words scores 100% on variable copying and names, so a high probe score there proves nothing [7]. **Pitfall 3, readable is not used:** for object tracking and names, probes read the state far better from ε than from $\hat{x}$ (84%

vs 48%; 97% vs 20%), yet copying that token brings back at most 47%: the model does not use it there yet.

B.6 Verdict on the original hypotheses

Table 10: What the pilot says about each original hypothesis.

H1 (error carries the state)	**Rejected** for every task: $\hat{x}$ beats ε at every layer.
H2 (low-dimensional structure in the error)	**Not supported as stated**: the state can be read from ε but is not used there, and for two tasks the prompt's words alone give it away.
H3 (a few directions repair)	**Supported, narrowly**: one direction closes about half of object tracking's gap, two close 58% of the running sum's; the directions are task-specific. This became the project's claim.

B.7 Caveats

- One model (gemma-2-2b), one SAE family (Gemma Scope), synthetic tasks; width tested at layer 12 only.
- Object tracking is the hardest task for the model: in the main runs only 27% of generated candidate pairs passed its pair filter (40% to 100% for the others), so its pairs are the most selected. Figure 2 uses the capability study's sample and criteria, hence its higher 84%.
- The names control's late-layer loss (24% to 36% at layers 18 to 20) is spread over many directions: part of the late-layer loss is a general SAE weakness.
- The 131k and 262k SAEs also use more active features per token (124 to 139 vs 78 to 89 at 16k), so part of the shrinking gap may come from that rather than from width.
- The binding explanation is a hypothesis, and the multilingual work is a plan with no results yet.

C Multilingual design

Table 11: Multilingual hypotheses (criteria as in Table 1).

ID	Hypothesis	Measure	Supported if
M1	Lower fidelity on Hindi text	fidelity on generic text, per language	at least 3 points below English (98% to 99%)
M2	SAEs miss more task state in Hindi	gap and specificity per language	larger gap than English, non-overlapping intervals
M3	The error direction transfers across languages	direction learned in English, used on Hindi	closes at least half as much as Hindi's own

Languages: English (baseline); Hindi (Devanagari; the best-resourced Indian language, most likely to work); Hindi in Latin script without English words, and Hinglish (Latin script, code-mixed), which together separate script from vocabulary; Tamil or Bengali if the model can solve the tasks; French as a high-resource non-Indian control.

Templates. The answer must be the last token, so verb-final Hindi uses a question plus "answer" format. All examples are *illustrative, to be checked by a native speaker and against the tokenizer*:

- Running sum (Hindi): शुरुआत 2 से करें। 3 जोड़ें। 1 जोड़ें। 2 जोड़ें। कुल = → 8
- Object tracking (Hindi): सिक्का डिब्बे B में है। चाबी डिब्बे E में है। चाबी डिब्बे C में चली जाती है। चाबी किस डिब्बे में है? डिब्बा → C
- Names (Hinglish): `Jab Rahul aur Priya market gaye, Priya ne ek book di. Book kisko mili? Answer:` → Rahul
- Variable copying is code-like and identical in every language: a language-neutral control.

Constraints: prompts A and B need the same number of tokens and a one-token answer; many Hindi names split into several tokens, so names are picked from single-token ones (the pipeline already drops and counts pairs that break this). **Steps:** (1) capability check per language (at least 30% usable pairs; failing is itself a finding); (2) fidelity on IndicCorp v2 and FineWeb-2 text (M1); (3) patching, gap, repair and specificity (M2); (4) the English direction used on Hindi (M3).

D Glossary

Token	a word or word piece; the unit the model reads
Layer	one of gemma-2-2b's 26 processing stages
Internal signal (residual stream)	the model's running representation at one layer: 2304 numbers per token
Sparse autoencoder (SAE)	a tool that rewrites the internal signal as a few active, human-readable features
Features, $\hat{x}$	the SAE's reconstruction from its active features ("SAE parts" in the appendix figures)
SAE error, ε	$x - \hat{x}$: what the SAE does not explain ("dark matter", informally the leftovers)
Width	how many features an SAE can choose from (16k = 16,384 up to 262k)
Active features (L0)	how many features are switched on for one token
Reconstruction fidelity	fraction of next-token performance retained when $\hat{x}$ replaces x (CE loss recovered)
Prompts A and B	two prompts that differ in one step, so the right answer differs ("clean" and "corrupt")
Activation patching	run on B but replace part of its activation with A's at one layer
Logit, NLDR	a logit is the model's raw score for a next token; NLDR is the effect recovered (0 to 1)
Gap	the share the features miss: one minus NLDR of copying $\hat{x}$ only
Direction	one fixed pattern in the 2304 numbers; "N directions" = an N-dimensional slice of the signal
Unsupervised / supervised directions	top principal directions of the error / directions that best separate the answers
Specificity score	share closed by same-task directions minus the best share from any other source
Probe	a simple classifier trained to read a value (e.g. the box) from the signal
Binding	how a model links an entity to its attribute (an object to its box)
IOI	"indirect object identification", the names task [14]
Hinglish	Hindi written in Latin script and mixed with English words
95% CI	bootstrap confidence interval: the range the result would likely fall in on new test pairs